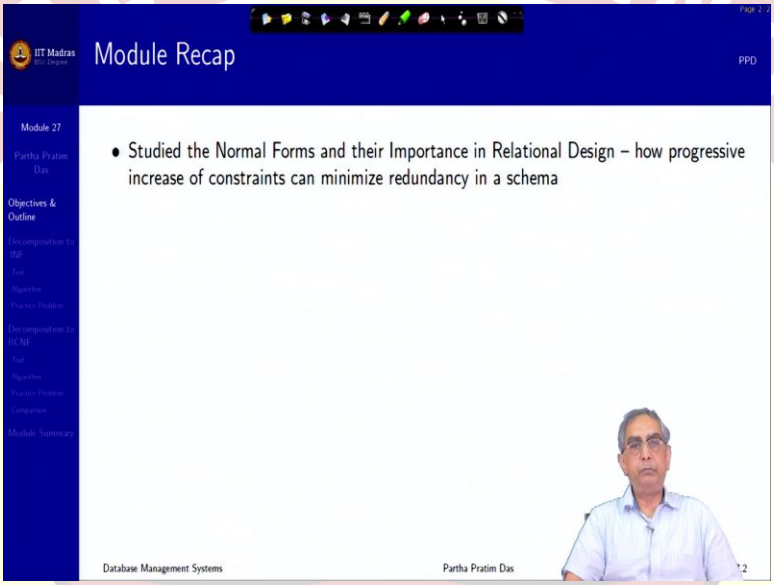
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Instituted of Technology, Kharagpur
Relational Database Design/7: Normal Forms

Welcome to Module 27 of database management systems course in IIT Madras online B.Sc. programme.

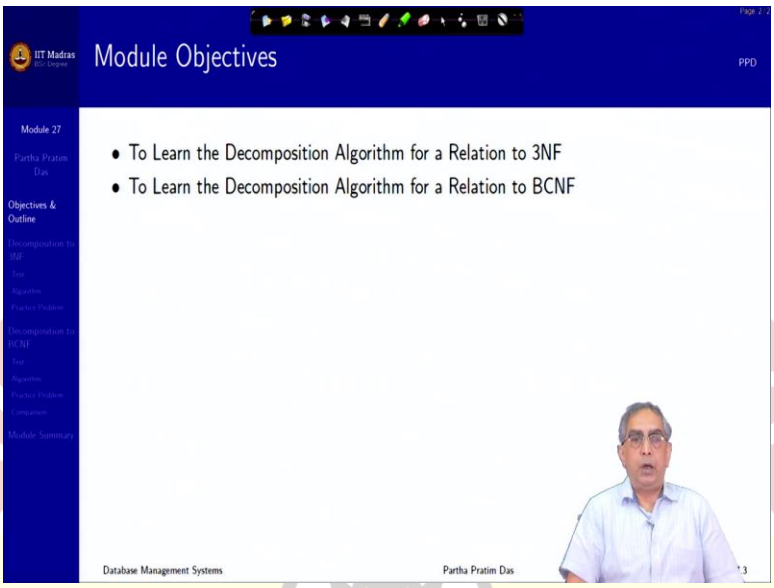
(Refer Slide Time: 00:35)



As you know, we have been discussing about relational database design different aspects of that going through the development of functional dependency theory and then defining various forms of normalisation or normal forms rather, first normal form, second normal form, third normal form to achieve better reduction of redundancy, we have also mentioned Boyce-Codd normal form earlier.

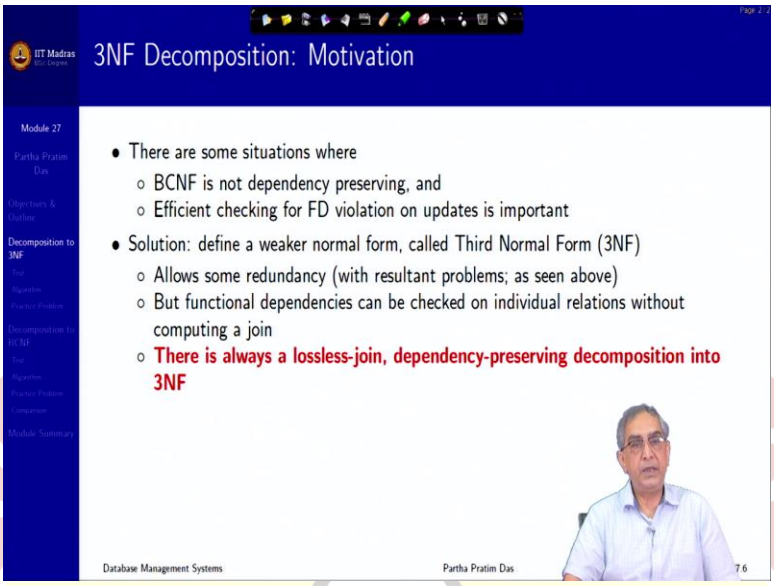
So, just to quickly recap first normal form is one where all attributes are atomic, second normal form is first normal form in addition would not allow partial dependencies and third normal form is in second normal form and in addition will not allow transitive dependency, Boyce-Codd normal form is defined in a little different way, which is a restriction on the third normal form.

(Refer Slide Time: 01:33)



Now, what we will do in this module is to study algorithms that can decompose a relation, a schema which is not there in the third normal form, decompose it into several two or more relational schema such that each relational schema can be in third normal form. Similarly, we will do that for the Boyce-Codd normal form we will see the algorithms and we will see the properties that hold.

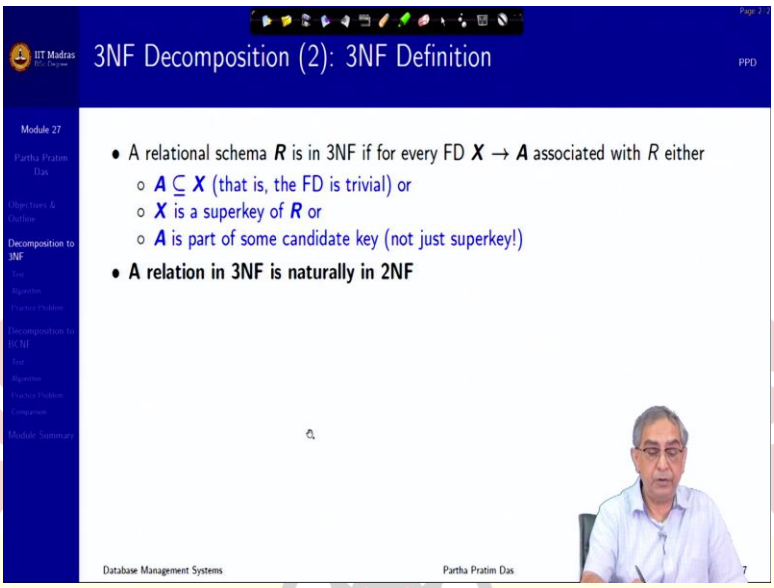
(Refer Slide Time: 02:05)



So, first decomposition to third normal form. So, why do we, why are we looking at third normal form. We have shown that with counter examples that the Boyce-Codd normal form which is very good in reduction of redundancy and is simple to implement in the decomposition also is not dependency preserving, that is there may be some dependencies which actually hold but cannot be tested only on the decomposed relations.

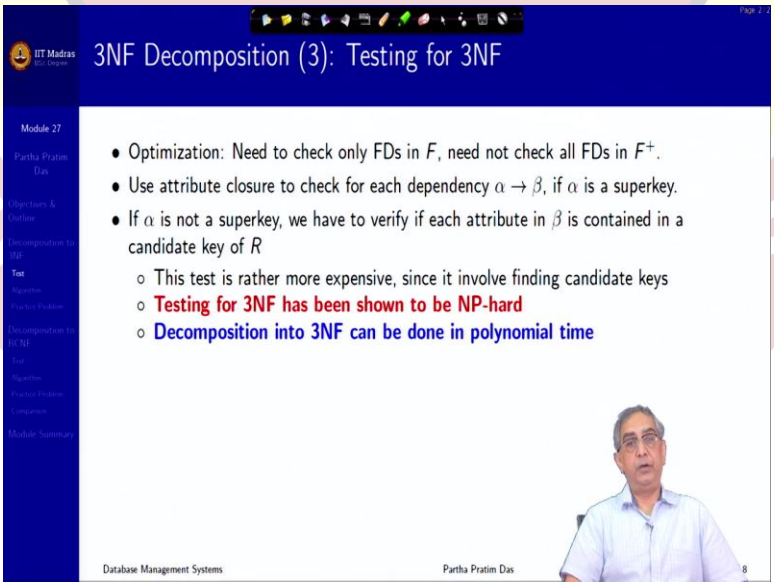
So, efficient checking of FD violation on updates is important which is very difficult with in Boyce-Codd normal form to do because you have to take a natural join. So, the solution to that is to define a weaker normal form, which will not reduce redundancy to the extent Boyce-Codd can do, but it will have the desirable property that all dependency preservation can be ensured along with the lossless joint property that BCNF also has. So, that is the reason we want to do third normal form.

(Refer Slide Time: 03:18)



Just to repeat a third normal form a relation is in third normal form if for every functional dependency X determining A, either A is a subset of X that is it is a trivial dependency or X is a super key of R these two actually are the conditions for the Boyce-Codd normal form and by relaxation, we provide a third condition that A is part of some candidate key not just super key, but just some candidate key naturally 3NF is also in 2NF we have seen already.

(Refer Slide Time: 03:52)

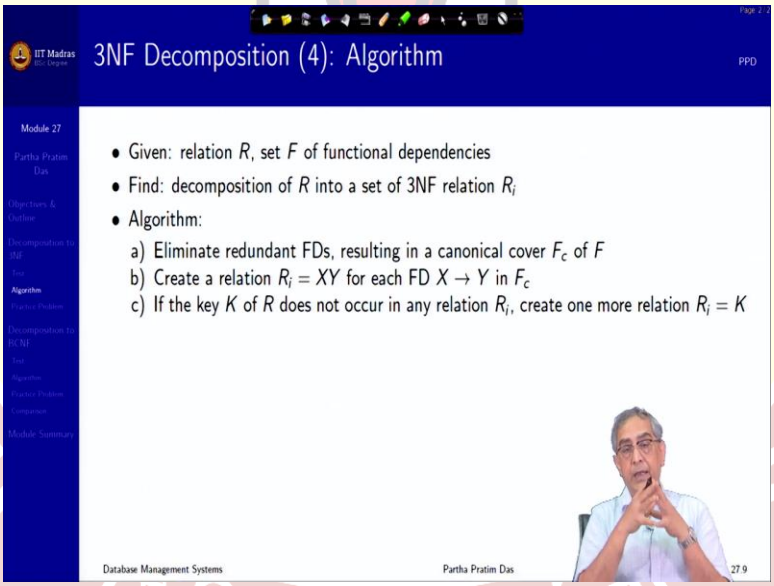


Before we go into discussing how to decompose a relation, let us see, can we check if a given relational schema is already in third normal form or not. So, the problem in this is first is we

need to, do we need to check that the whole of the closure set of functional dependencies that is a whole set of closure set a functional dependencies is a problem. So, fortunately, we do not need to check on F+ it is enough to check on F, we can use attribute closure to check for each dependency alpha determining beta if alpha is a super key this part is easy. If alpha is not a super key, then we have to verify that each attribute in beta is contained in some candidate key.

This particular test is very expensive, because you have to consider all candidate keys that are possible and there could that could be really large and it has been proven that testing for third normal form is NP hard. If you remember your algorithms course you will know that there is a class of problems known as NP hard problems for which there is no polynomial time known algorithm to solve the problem. So, certainly, that is not a testing for 3NF is not a solution. But fortunately, we can decompose a relation into third normal form in polynomial time and that is what we are going to learn here.

(Refer Slide Time: 05:33)



So, the algorithm is very simple, given a relation R and a set of functional dependencies F, we want to decompose R into n number of smaller relations Ri's. Now, how will you do that? We will first eliminate redundant functional dependencies rather, I should say that, we will compute the canonical cover, we have seen how to do that, that is not difficult.

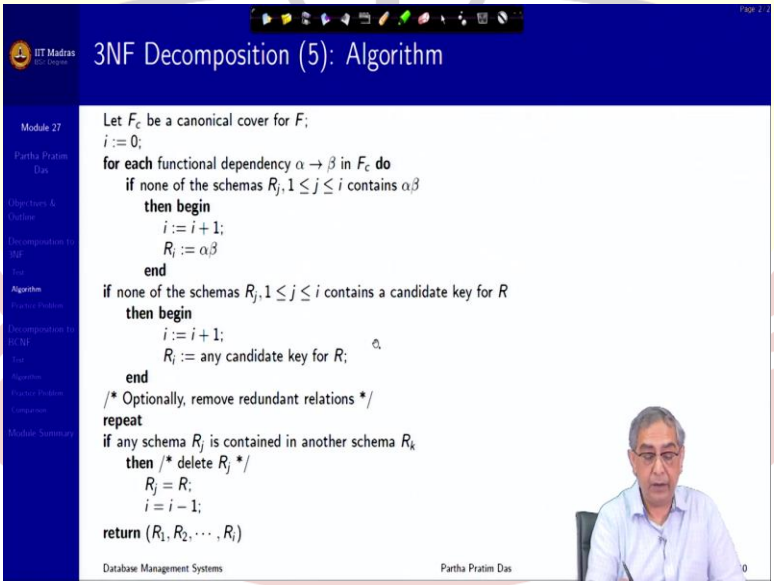
So, that will be kind of a minimal representation of the set of functional dependencies and for each functional dependency in the minimal cover in the canonical cover Fc. Let us, say if there is

a functional dependency X determines Y, then we will create a relation decomposed relation XY that is take the union of attributes in X, take the union of attribute with the attributes of Y and create a new relational scheme Ri.

And in this way, as many functional dependencies are there in the canonical cover, we will get those many decomposed relations, if the key of the relation does not occur as a subset of any of these Ri's then we will add one more relation which is K. So, three simple steps, canonical cover take every dependency, take union of left and right inside make it a schema check if there is any of these which covers K the key if it does not then include K also.

There is proof of course, which we are not going into you can easily do that it is not difficult to do that this will be in all each and every decompose relation created in this way will be in third normal form. Now, one intuition is very clear is because we are creating relational schemas as X determines Y giving XY, so naturally in X in Ri, which is XY, X is the key. Now, we will have to see what are the other dependencies which might hold over that set as well. Now, since it is minimal, naturally, there will be only one dependency with X on the left hand side.

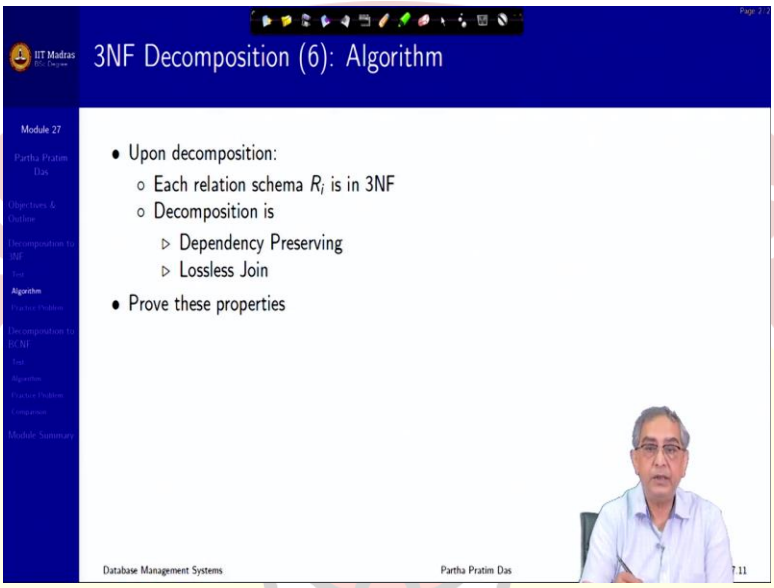
(Refer Slide Time: 08:21)



Now, I will not go through each and every step of this because it is not so easy to understand on the readout, but this is exactly as I have intuitively explained, this is the actual pseudo coded algorithm for doing the decomposition into third normal form using the same strategy. So, initially, you are computing the minimal cover, checking for the candidate key and removing in

the process of removing the redundant relations and finally, you generate the R1, R2, Ri the different schemas corresponding to the different functional dependencies that have been retained in the canonical cover.

(Refer Slide Time: 09:09)

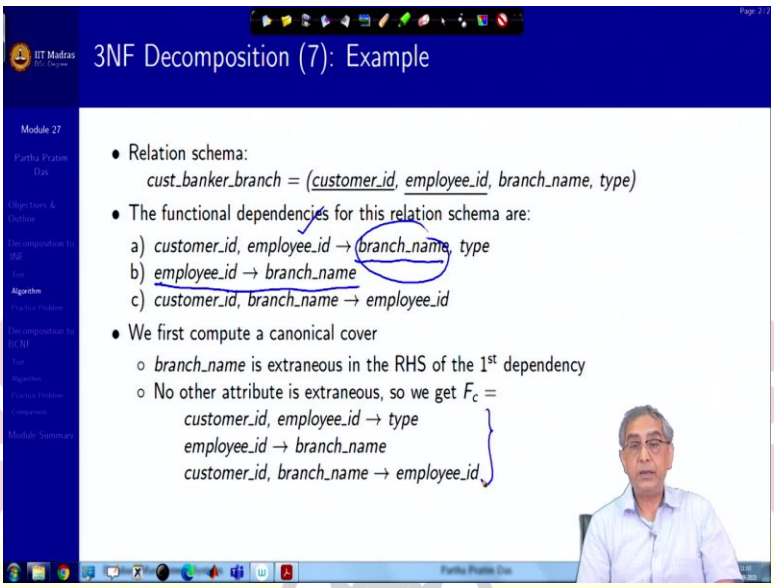


So, upon decomposition what you will have is that each relational schema in Ri is in third normal form which is easy to see. But most importantly, this decomposition is dependency preserving and also lossless join. It is dependency preserving is also easy to see because you have created a schema for each and every functional dependency in the minimal cover.

So, each and every functional dependency in the minimal cover will be checked, can be checked in the corresponding decomposed relation. So, if all functional dependencies can be checked, then naturally there closer set the closer set of the minimal cover which is a closer set of F can also be checked.

And it is a lossless join that you can prove by very simply taking two relations and working out I mean decomposition into two relations using three attributes you can show that in every step of this decomposition which you have now done in one go, you are actually preserving the lossless join property that is the intersection between any two decomposed relation decides the is a key of one of the decomposed relations you can prove this in these properties at length and get confidence (())(10:56).

(Refer Slide Time: 10:56)

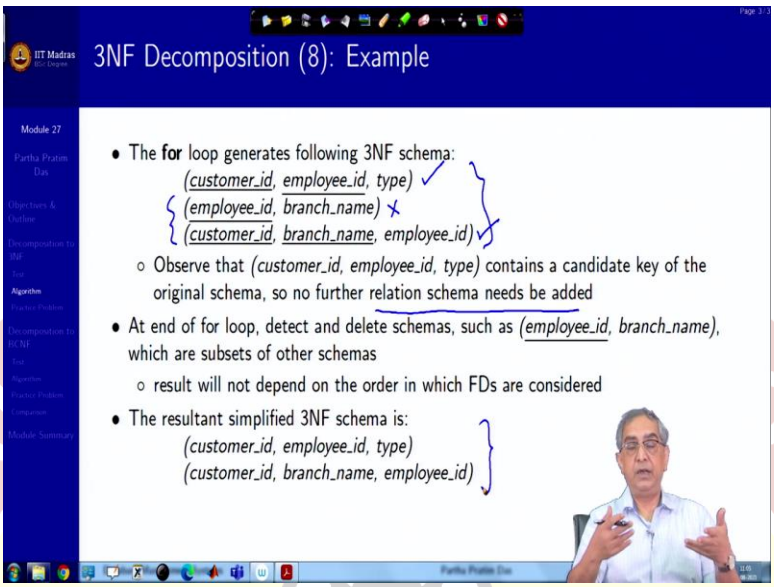


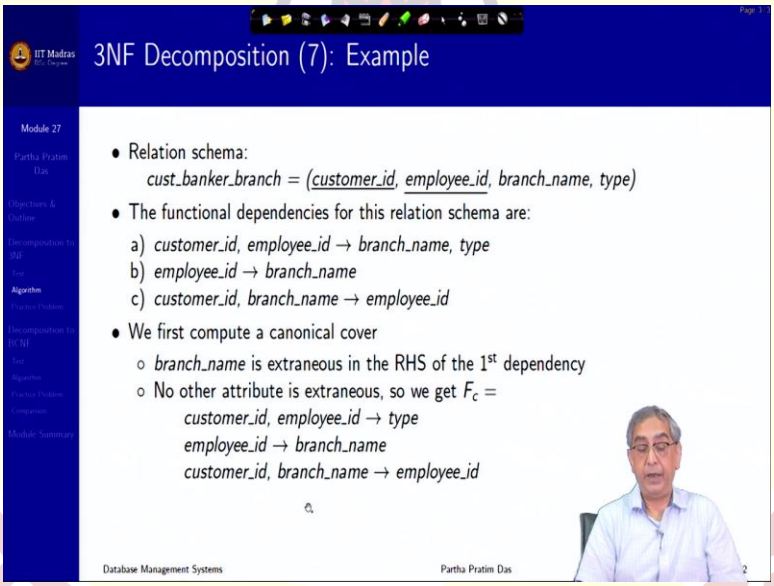
I will quickly take some examples, say let us there is a relational schema, customer banker branch which has customer_ID, employee_ID, branch name and type obviously, customer_ID and employee_ID together is the key. What are the functional dependencies customer_ID and employee_ID will obviously decide the branch name and type, then employee_ID employee works in one branch only.

So, employee ID will decide the branch name and the customer if the goes to a particular branch, the given the customer the employee ID is given that is so this employee is kind of the relationship manager for the customer, who kind of caters to the customer on behalf of the bank. So, if you, if I know the branch the customer goes to then the customer has a fixed employee who caters to him.

So, given this, we can first compute a canonical cover and you can see very easily that branch name is an extraneous attribute. Why is it extraneous because employee ID determines branch name. So, you do not need to include it here you have employee ID on the left hand side. So, you remove that and you get a simple covered which are these three functional dependencies.

(Refer Slide Time: 12:24)





Once you have got that, then the loop will generate. Let me, I should have also given the functional dependencies here, so we will just have to remember it is customer_ID, employee_ID determining type, employee_ID determining branch name and customer_ID brands name determining employee_ID.

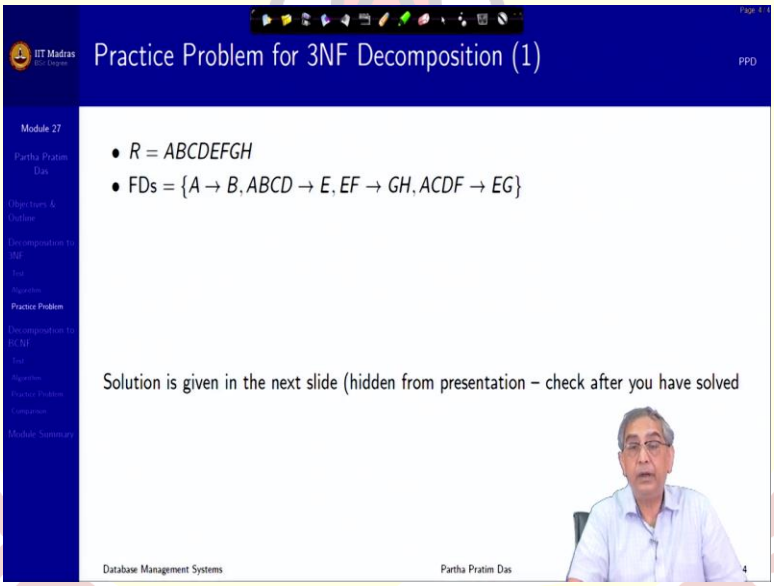
So, what we will do for each of these, we will take the union on the left hand right hand side and these are the corresponding three relations that we get for the schema. The keys are marked which are easy. Now, you have in the third, if you look in the first decomposed relation, you will observe that customer_ID and employee_ID are the keys were the original relation and that is

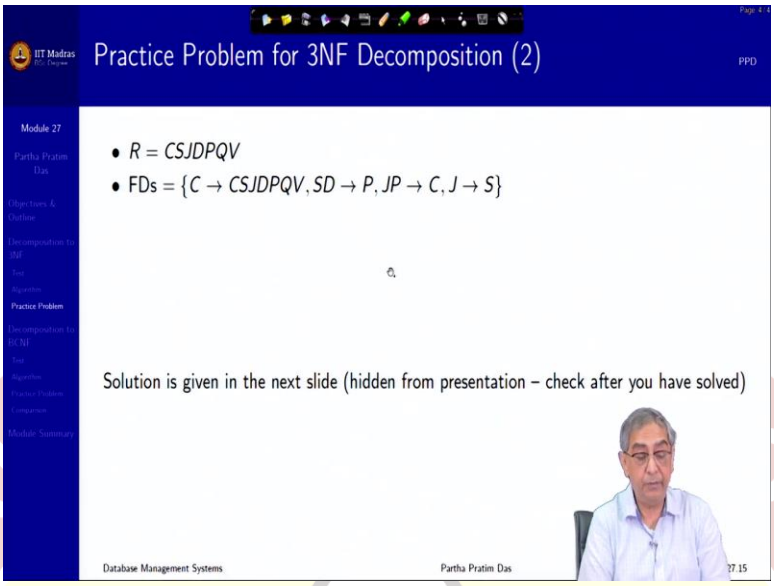
contained in this component. So, we do not need to add customer_ID, employee_ID as a separate decomposed relational schema.

So, no further schema need to be added. You also make another observation that if you look into second and third, second actually is a subset of the third. It is actually a subset of the third. So, if we have that, then naturally, we will keep this and we do not need this because in any case, we will have all that information.

So, with this, we finally get 3NF schema which is customer_ID, employee_ID type and customer ID branch name, employee_ID. So, this is the process through which it goes and mind you this is not impacted by the order in which you process the FD's whatever is the order, you will get the same result for this from this minimal set.

(Refer Slide Time: 14:27)

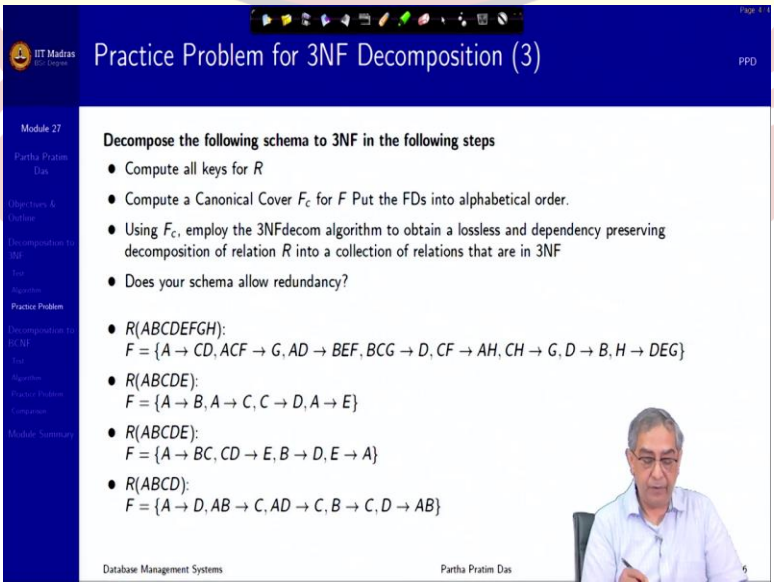




So, I have included here to practice problems for which the solution is given. But I have hidden it from the presentation. So, that when you look at the presentation slides you will find the solution in the next slide. But I am not showing it here. So, that you can first take a dig and saw try to solve it yourself and then check with the solution.

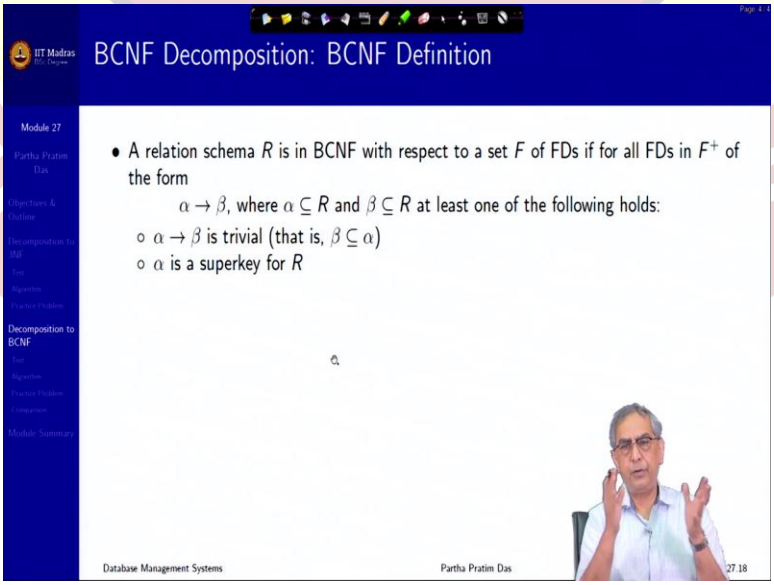
So, here is a big relation with the 8 attributes and there are 1, 2, 3, 4 different functional dependencies, you will have to do the minimal cover and all that and similarly, another problem is given here with 7 attributes and some 4 functional dependencies, you will also have to solve that and can check the solution from the presentation.

(Refer Slide Time: 15:21)



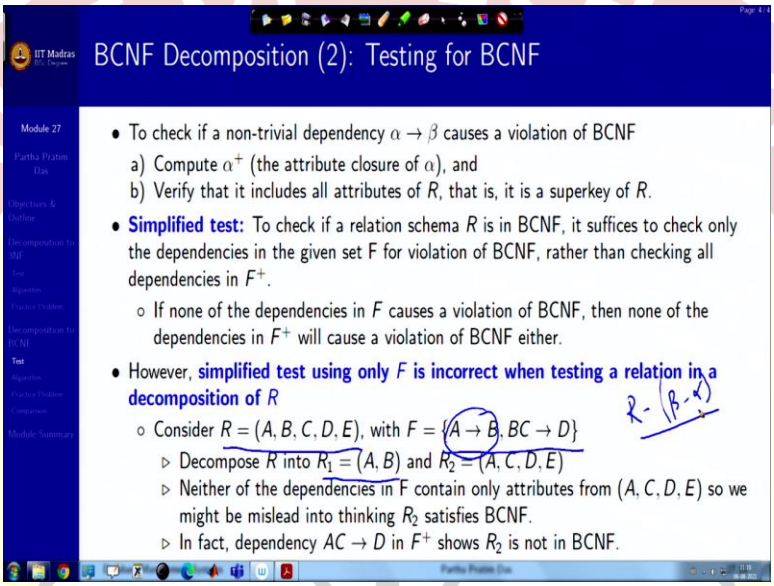
And finally, here are some more practice problems for which the solution is not given in the presentation and you should compute all keys for every relation compute the canonical cover using the canonical cover use the 3NF that is decomposition algorithm to obtain a lossless join and dependency preserving decomposition and since it is 3NF you know it will have redundancy because it is weaker than BCNF. So, after you have done the design, you will have to comment on does your schema allow redundancy and if it does, then you should create a at least one example showing that where the redundancy is coming from.

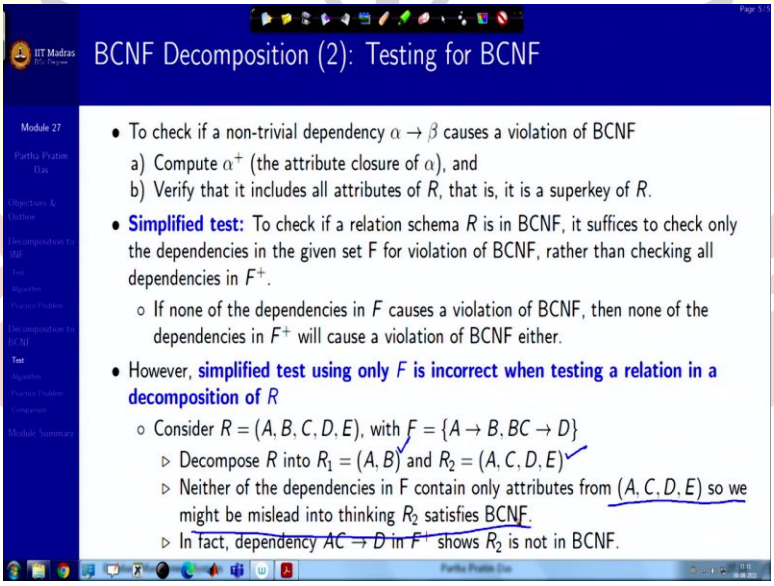
(Refer Slide Time: 16:17)

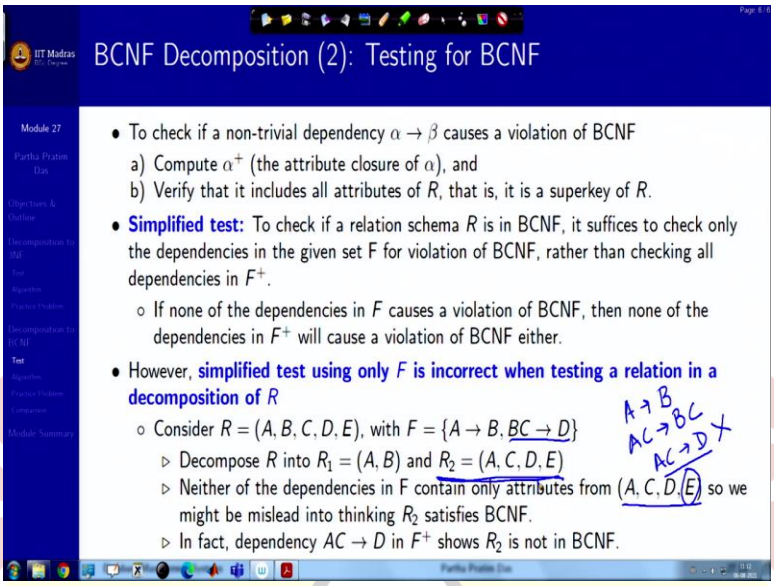


So, that was about decomposing into third normal form let us move into the Boyce-Codd normal form. We have talked about Boyce-Codd normal form several times already but just formally speaking, what the decomposition issues are. So, this is the definition of the Boyce-Codd normal form where every functional dependency alpha determining beta is either trivial or has a left hand side alpha which is a super key for R very simple definition.

(Refer Slide Time: 16:47)







Now, again the same question as of 3NF can you check if a relational schema is already in Boyce-Codd normal form. Now, so what will you have to check that whether a non-trivial, trivial dependency is obviously trivial to check. But if there is a non-trivial dependency alpha determines beta whether it causes a violation of BCNF. How do you do that you can compute α+ the closure of the attributes and verify that α+ is equal to R because it has to be the super key.

Now, you can do some simplification to check if a relational schema is in BCNF it is enough to check only the dependencies in the given set F you do not have to check for the entire F+. If there is no BCNF violation for the dependencies in F there will not be any violation for the dependencies that get included in the closure set, this is, I mean you can think and get convinced about it. Otherwise, you can look up further proof.

But when you are doing the simplified test using only F if you are testing a decomposed relation to be in BCNF if you have a decomposed relation that you cannot check simply by using F you will have to take the F+ as a whole and from F+ you will have to take out only those functional dependencies, which has attributes available in that particular decomposition.

So, to just emphasise that point, we have given an example where the I am sorry there is a relational schema with 5 attributes and 2 functional dependencies. Now, we what is the key A determines B. So, and BC determines D. So, obviously, A does not determine everything. So, A is not a key. So, this violates.

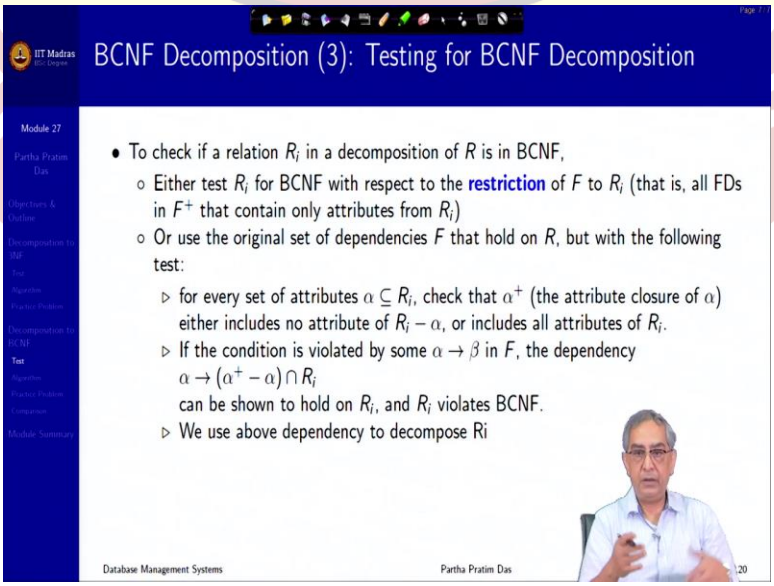
Now, if this violates then I have a decomposition as what did they say alpha and beta union is 1 and the other is where you take out alpha from beta and take that result set out from that is R minus beta minus alpha. So, if you do that here, you get one which is AB, the other is if you do beta minus alpha, you get only B.

So, from A, B, C, D you take out B and you get this. Now, if you try to check the functional dependencies in F on these decomposed relations, you will see that okay AB is satisfied on R1, B C D is not covered by any of the two decompositions. So, you might feel that it is it does satisfy.

So, you might be driven to think that this second decomposition is also in BCNF but that is not true because, what you will you can have is a sense A determines B you can have AC determines BC and BC determines D. So, AC determines D and if you project it, if you take this on this decompose relation then all attributes are there, so, this functional dependency holds on this particular decomposed relation, but AC is not a super key of that relation.

So, since AC is not a super key of the relation because E is there. So, since AC is not a super key of that relation, this particular functional dependency violates the BCNF for R2. So, you have decomposed into AB and ACD AB is in BCNF but ACD is still not in BCNF you will need to decompose it further. So, but checking with F you would have got a wrong impression that it is already in BCNF. So, that is the pitfall that you will have to avoid while testing for BCNF in this way.

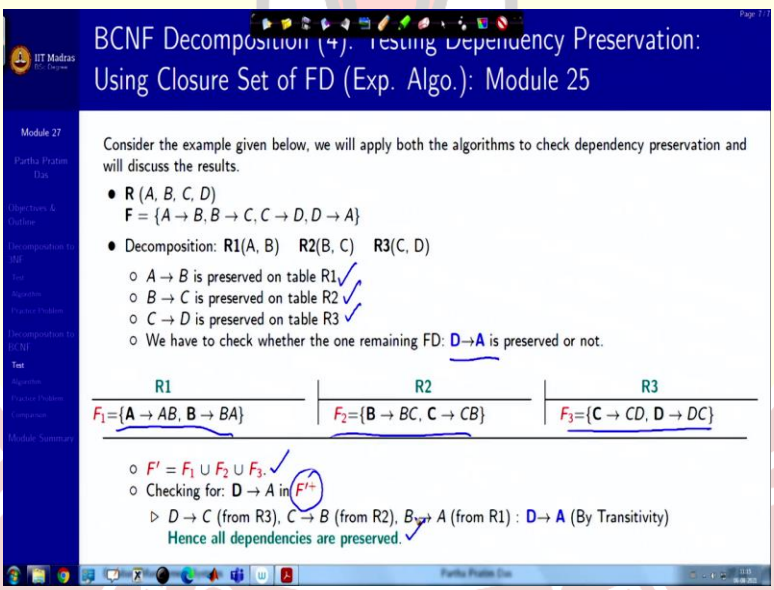
(Refer Slide Time: 21:47)



So, to check testing for BCNF decomposition, what you will have to do, you check if a relation Ri that is the decomposed relation Ri is in BCNF you have to first create the restriction that is either you will test Ri for BCNF with the restriction of F on to Ri that is compute F+ take all functional dependencies which has only the attributes in Ri and with respect to that set, you check whether Ri is in BCNF or naturally that is expensive or what you can do you can use the original set of functional dependencies, but test it using the attribute closure.

So, for every set of attributes alpha which is a subset of Ri, you check for α+ and either it includes no attribute from (Ri – α) or it includes all attributes of Ri. Now, if the condition is violated by some alpha determines beta, then this particular dependency (α+ - α) ∩ Ri will hold on Ri and Ri violates BCNF that is easy to see. So, we use this you know strategy to confirm that an Ri is in BCNF after decomposition or not, if it is not in BCNF we have to decompose it further.

(Refer Slide Time: 23:29)



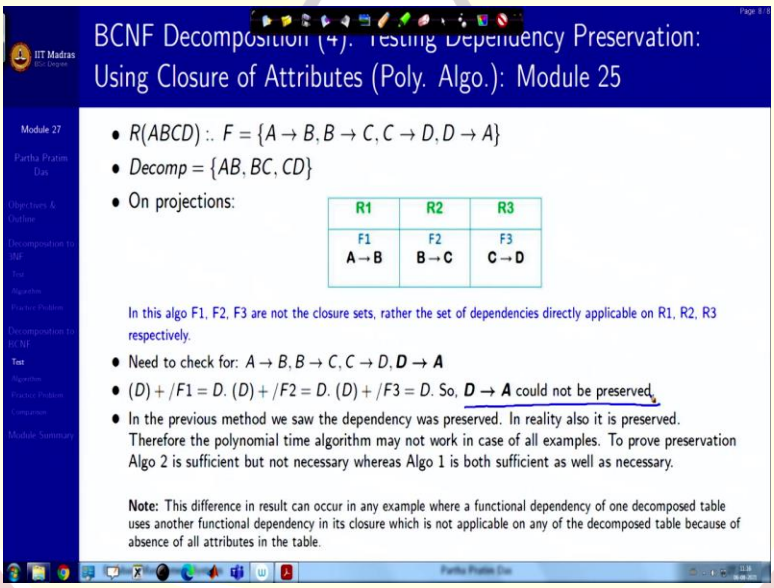
Now, at this point, I would also like to mention about another test that you might want to do suppose you have done the BCNF decomposition and you know by the theory of BCNF that the BCNF decomposition is actually lossless join. So, that is guaranteed, but you do not know if it is dependency preserving.

So, we have talked about two algorithms earlier while talking about dependency preservation to ensure whether a particular set of functional dependencies are preserved under a certain

decomposition. One was to compute in terms of the F+ and an exponential algorithms. So, if you follow that algorithm, you will see these are preserved and you have to check whether D A is preserved or not and you compute F1, F2, F3 which are actually computed from F+ and that is why they are expensive.

And you can after you take the union of them and compute the closure it will be you can clearly see that D actually functionally determines A if A determines B is satisfied in R1, B determined C satisfied R2 and C determines D is satisfied in R3. This is just simple transitivity and will show that. So, all dependencies are actually preserved. So, this decomposition is dependency preserving.

(Refer Slide Time: 25:10)



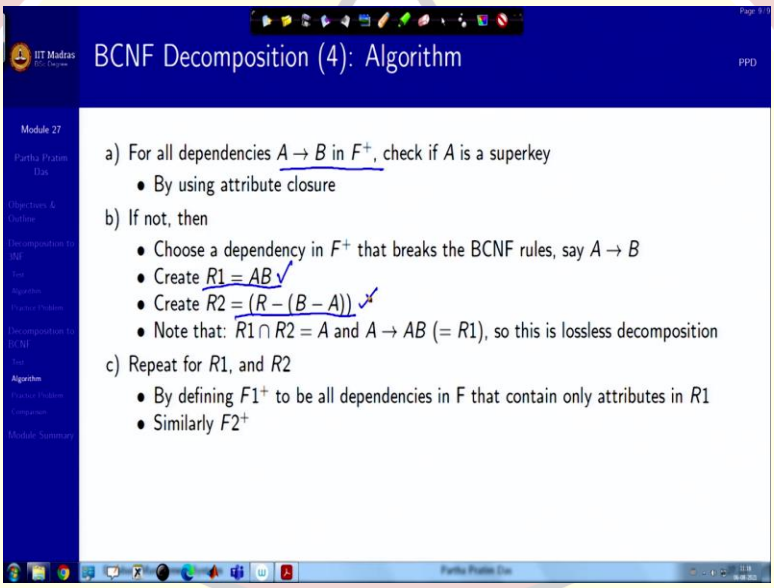
Unfortunately, naturally this is an exponential time algorithm. So, you will not often be able to run such kind of an algorithm. So, you will run the polynomial time algorithm that we talked of where we do not take the closure set the, but rather the set of dependencies that are directly applicable on R1, R2, R3 in a majority of cases this will give you correct result, but in this case, because there is cyclic dependency A determines B, B determined C, C determines D, D determines A and by cycled D determines B, D determines C all these come up.

But when you take three individually and do not take the closure, you will not be able to see this fact and since you cannot see this fact, so, the polynomial time algorithm will not preserve we will say that D determines A is not preserved. So, what it means is the polynomial time

algorithm is somewhat conservative in the way that it will not give you a false positive which means that if A decomposition is not dependency preserving, it will never give you that it is dependency preserving.

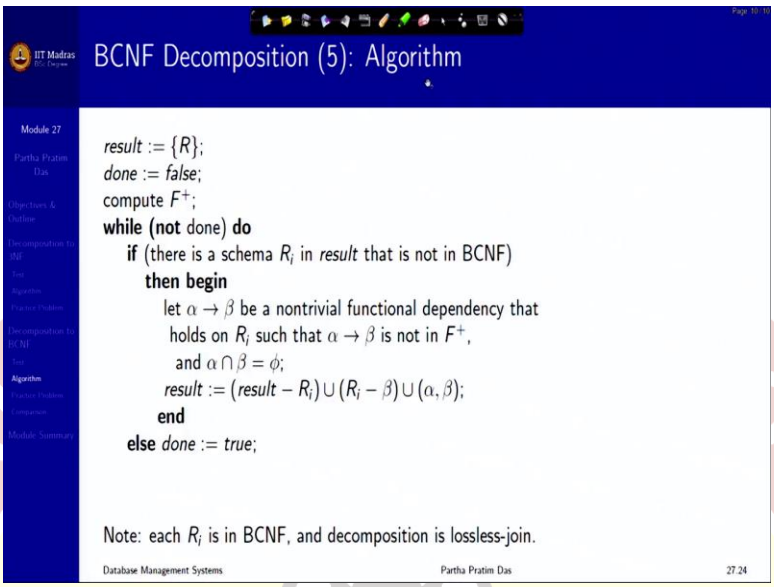
But it can give you a false negative that it says that it is not dependency preserving, but even then, it may actually be dependency preserving. The exponential algorithm will always give you the correct answer, but it is exponential. So, that (())(26:47) of most often we may not need to use it or maybe the polynomial time algorithm itself will always give you the truth results, but this is the difference that you will have to keep in mind in terms of the testing algorithm.

(Refer Slide Time: 27:05)



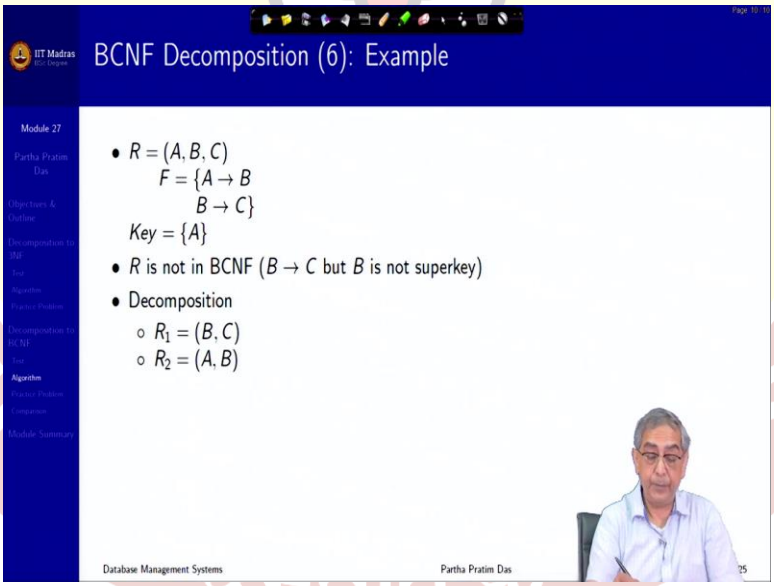
So, we have basically talked about three tests one as a testing for BCNF of the original relation, testing of BCNF for the decomposed relations and testing of functional dependency preservation of a BCNF decomposition. Now, the algorithm we had discussed earlier to it is very simple, if A determines B in F+ violates the BCNF condition, then you create one relation which is A union B and the other relation says this by construction satisfies the lossless join property and then you will have to this is in BCNF or this is in BCNF you will have to check each one of them and do this whole process of as we have discussed checking for BCNF for decomposed relations and if anyone is not then you will have to decompose that relation again.

(Refer Slide Time: 28:09)



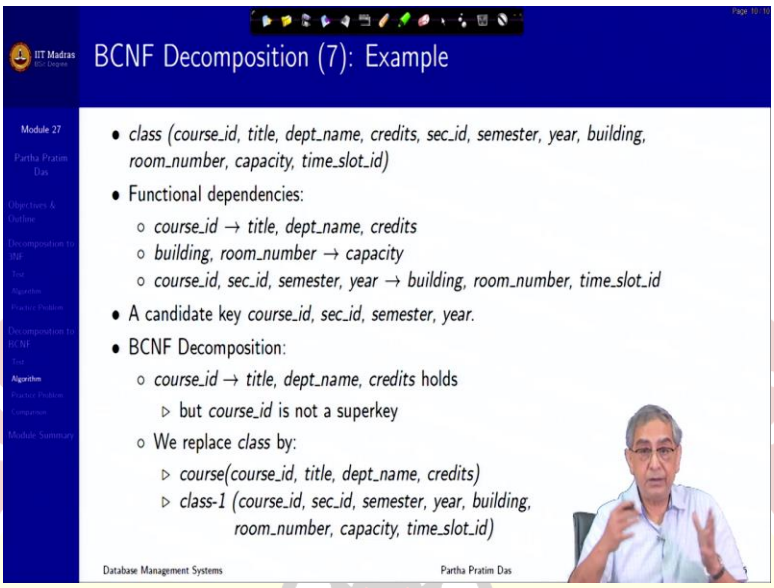
So, here is the algorithm put in pseudocode terms, you can go through that and understand it in detail.

(Refer Slide Time: 28:21)



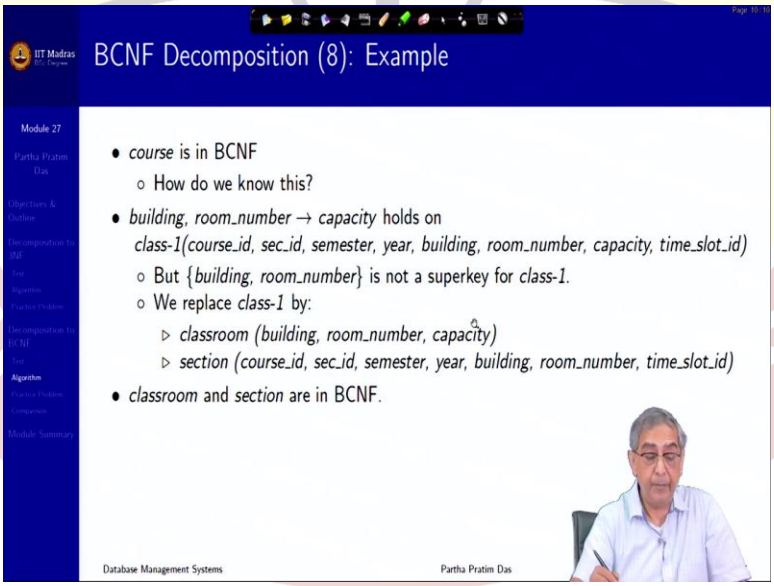
So, very simple another example is A determines B determines C, the key is A so B determines C is not in BCNF. So, you will decompose it as BC and after you have done that the other decomposition would be AB which is obvious.

(Refer Slide Time: 28:43)



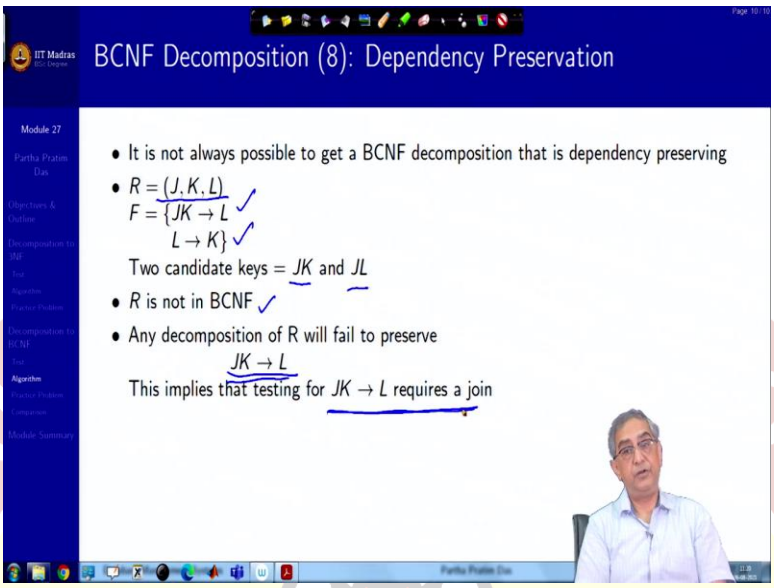
Here is another example worked out for class and course_ID and so on, which you can work through and understand for yourself, it is pretty simple, but I have included just so that you can get more and more familiar.

(Refer Slide Time: 29:00)



So, this is the workout of this example.

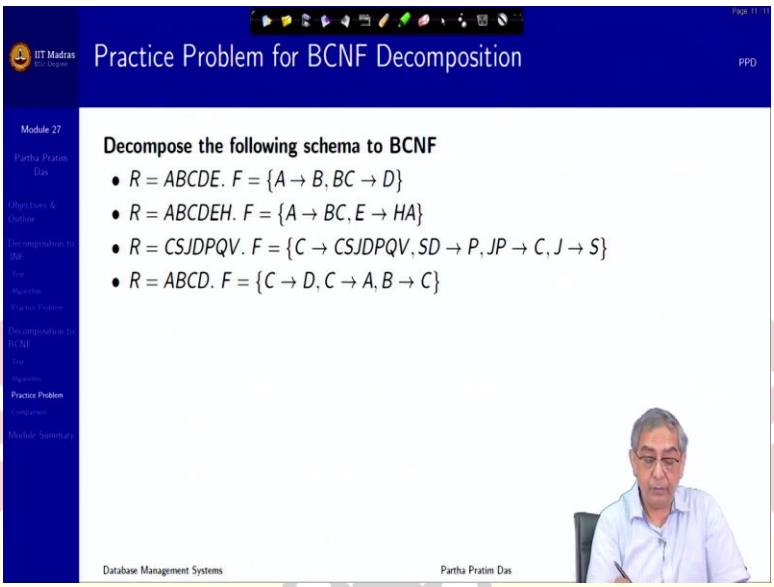
(Refer Slide Time: 29:03)



Now, here is I think we mentioned it earlier also here is just an example of a BCNF decomposition that does not preserve the dependencies. So, you can see that there are 3 attributes and 2 functional dependencies. There are 2 candidate keys J K and J L. So, R is not in BCNF and you will decompose R as you decompose R naturally this functional dependency which has all the tributes will not occurred in any of the decomposed relations.

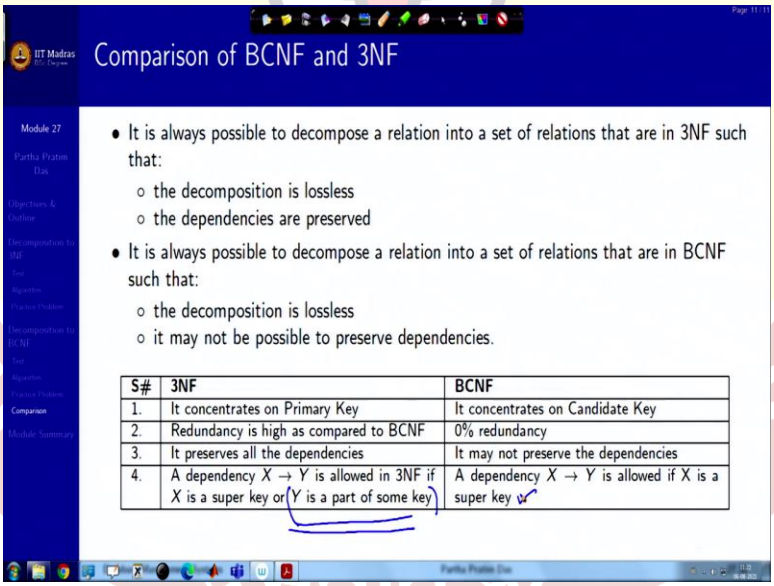
So, therefore, this decomposed BCNF form will not support the preservation of the dependency and therefore to check for this particular functional dependency we will always need to do an expensive join and there are several cases that and that as I said is a motivation for using 3NF.

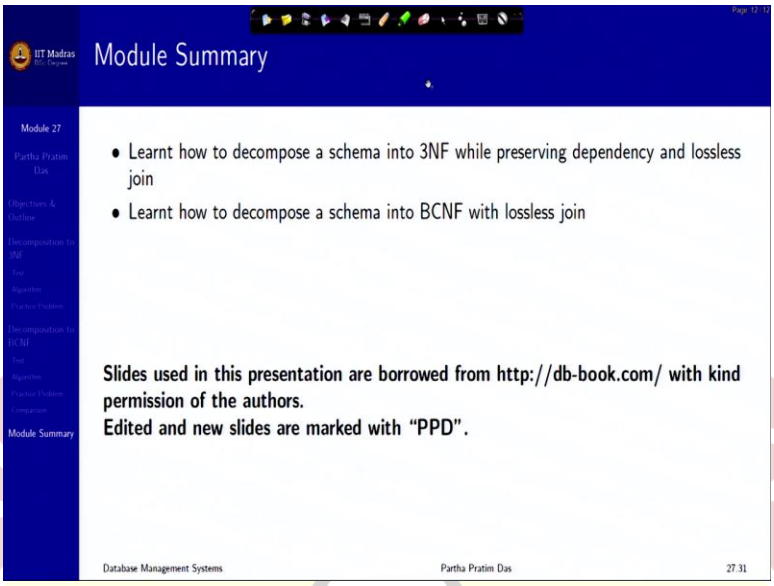
(Refer Slide Time: 30:08)



We have also included some practice problem, further practice problems for you, so that you can work them out and get confident about BCNF decomposition.

(Refer Slide Time: 30:20)





Finally, just to summarise where we stand in terms of decomposition, what we have learned is it is always possible to decompose a relation into a set of relations that are 3NF such that decomposition is lossless and dependencies are preserved. But the catch being that you have some redundancy, more redundancy left and the other that we have learned it is always possible to decompose a relation into a set of relations that are in Boyce-Codd normal form that decomposition is also lossless, but it may not preserve all the dependencies.

So, here is a just a comparison of the two normal forms. So, in 3NF you have a focus on the primary key, BCNF works with candidate key, BCNF have zero redundancy with respect to the scope that we are looking at, with respect to the functional dependencies. But 3NF will have some redundancy compared to BCNF, 3NF preserves dependencies, all dependencies BCNF may or may not.

And dependency X determines Y is allowed in 3NF if X is a super key or Y is part of some key. See this all part is important for 3NF which is the relaxation just remember that whereas in BCNF, it will have to be super key. So, that is the comparative scenario of 3NF and BCNF with this we come to the close of this module. Thank you very much for your attention. We will meet in the next module and continue further.